

**EXERCISE 11.5**

Think about how `replicateM` will behave for various choices of `F`. For example, how does it behave in the `List` monad? What about `Option`? Describe in your own words the general meaning of `replicateM`.

There was also a combinator for our `Gen` data type, `product`, to take two generators and turn them into a generator of pairs, and we did the same thing for `Par` computations. In both cases, we implemented `product` in terms of `map2`. So we can definitely write it generically for any monad `F`:

```
def product[A,B](ma: F[A], mb: F[B]): F[(A, B)] = map2(ma, mb)((_, _))
```

We don't have to restrict ourselves to combinators that we've seen already. It's important to *play around* and see what we find.

**EXERCISE 11.6**

Hard: Here's an example of a function we haven't seen before. Implement the function `filterM`. It's a bit like `filter`, except that instead of a function from `A => Boolean`, we have an `A => F[Boolean]`. (Replacing various ordinary functions like this with the monadic equivalent often yields interesting results.) Implement this function, and then think about what it means for various data types.

```
def filterM[A](ms: List[A])(f: A => F[Boolean]): F[List[A]]
```

The combinators we've seen here are only a small sample of the full library that `Monad` lets us implement once and for all. We'll see some more examples in chapter 13.

11.4 *Monad laws*

In this section, we'll introduce laws to govern our `Monad` interface.⁶ Certainly we'd expect the functor laws to also hold for `Monad`, since a `Monad[F]` is a `Functor[F]`, but what else do we expect? What laws should constrain `flatMap` and `unit`?

11.4.1 *The associative law*

For example, if we wanted to combine three monadic values into one, which two should we combine first? Should it matter? To answer this question, let's for a moment

⁶ These laws, once again, come from the concept of monads from category theory, but a background in category theory isn't necessary to understand this section.

take a step down from the abstract level and look at a simple concrete example using the Gen monad.

Say we're testing a product order system and we need to mock up some orders. We might have an Order case class and a generator for that class.

Listing 11.6 Defining our Order class

```
case class Order(item: Item, quantity: Int)
case class Item(name: String, price: Double)

val genOrder: Gen[Order] = for {
  name <- Gen.stringN(3)
  price <- Gen.uniform.map(_ * 10)
  quantity <- Gen.choose(1,100)
} yield Order(Item(name, price), quantity)
```

A random string of length 3

A uniform random Double between 0 and 10

A random Int between 0 and 100

Here we're generating the Item inline (from name and price), but there might be places where we want to generate an Item separately. So we could pull that into its own generator:

```
val genItem: Gen[Item] = for {
  name <- Gen.stringN(3)
  price <- Gen.uniform.map(_ * 10)
} yield Item(name, price)
```

Then we can use that in genOrder:

```
val genOrder: Gen[Order] = for {
  item <- genItem
  quantity <- Gen.choose(1,100)
} yield Order(item, quantity)
```

And that should do exactly the same thing, right? It seems safe to assume that. But not so fast. How can we be sure? It's not exactly the same code.

Let's expand both implementations of genOrder into calls to map and flatMap to better see what's going on. In the former case, the translation is straightforward:

```
Gen.nextString.flatMap(name =>
  Gen.nextDouble.flatMap(price =>
    Gen.nextInt.map(quantity =>
      Order(Item(name, price), quantity))))
```

But the second case looks like this (inlining the call to genItem):

```
Gen.nextString.flatMap(name =>
  Gen.nextInt.map(price =>
    Item(name, price))).flatMap(item =>
  Gen.nextInt.map(quantity =>
    Order(item, quantity)))
```

Once we expand them, it's clear that those two implementations aren't identical. And yet when we look at the for-comprehension, it seems perfectly reasonable to assume that the two implementations do exactly the same thing. In fact, it would be